

Orca Paper

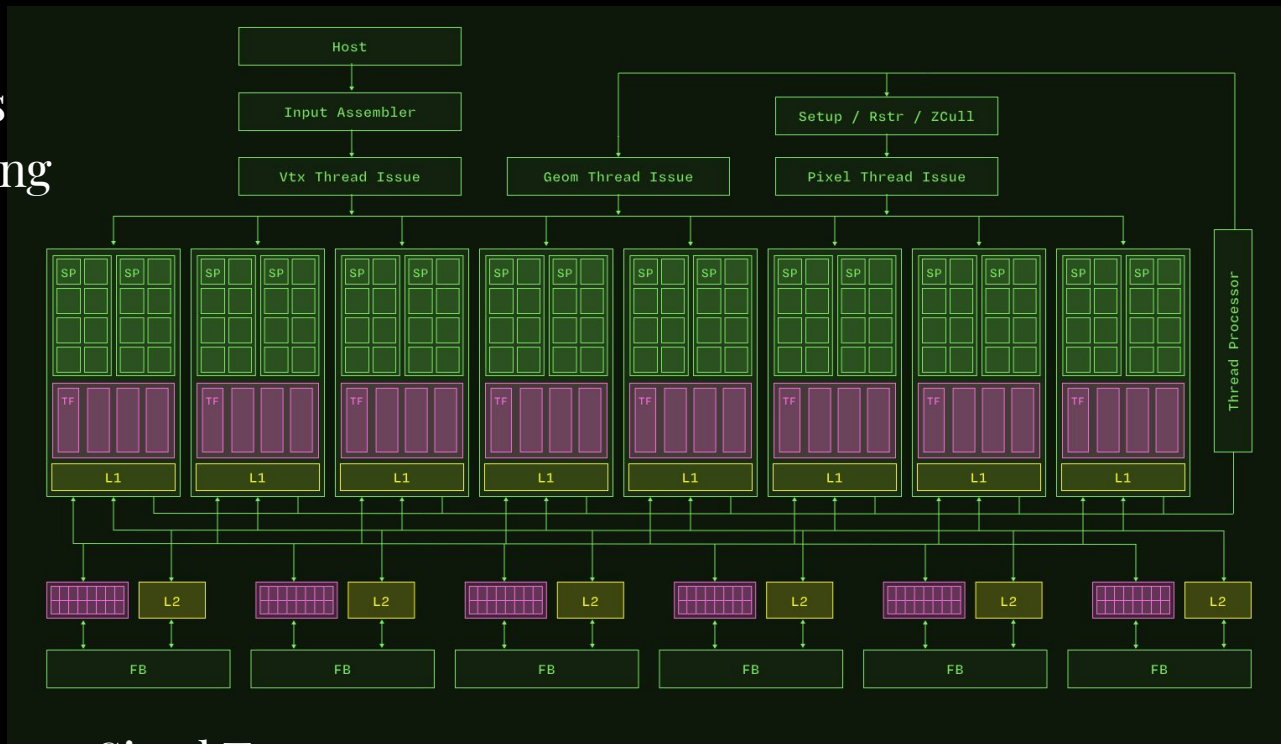
Pre-Req

How GPU works – Kinda

- Has Shit ton of cores
- Not Good at Branching

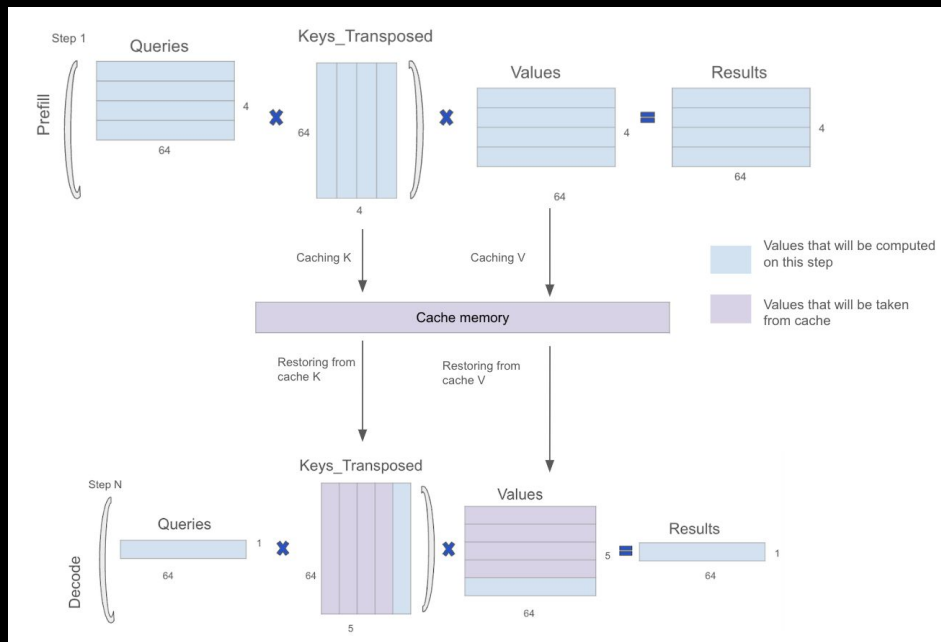
Note:

- Batching is Good, helps us make use of the entire GPU
- Batching Requires Same Sized Tensors

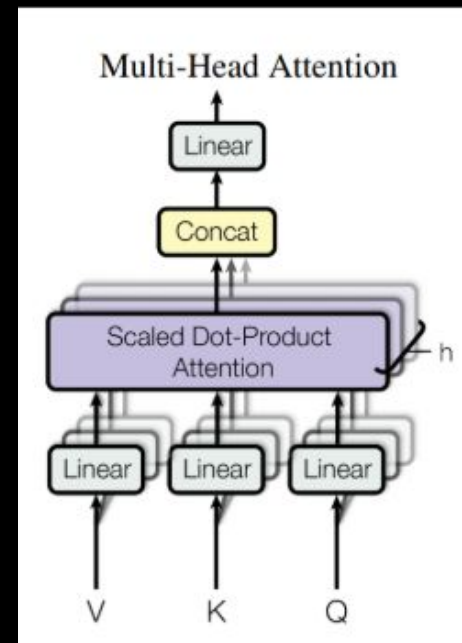
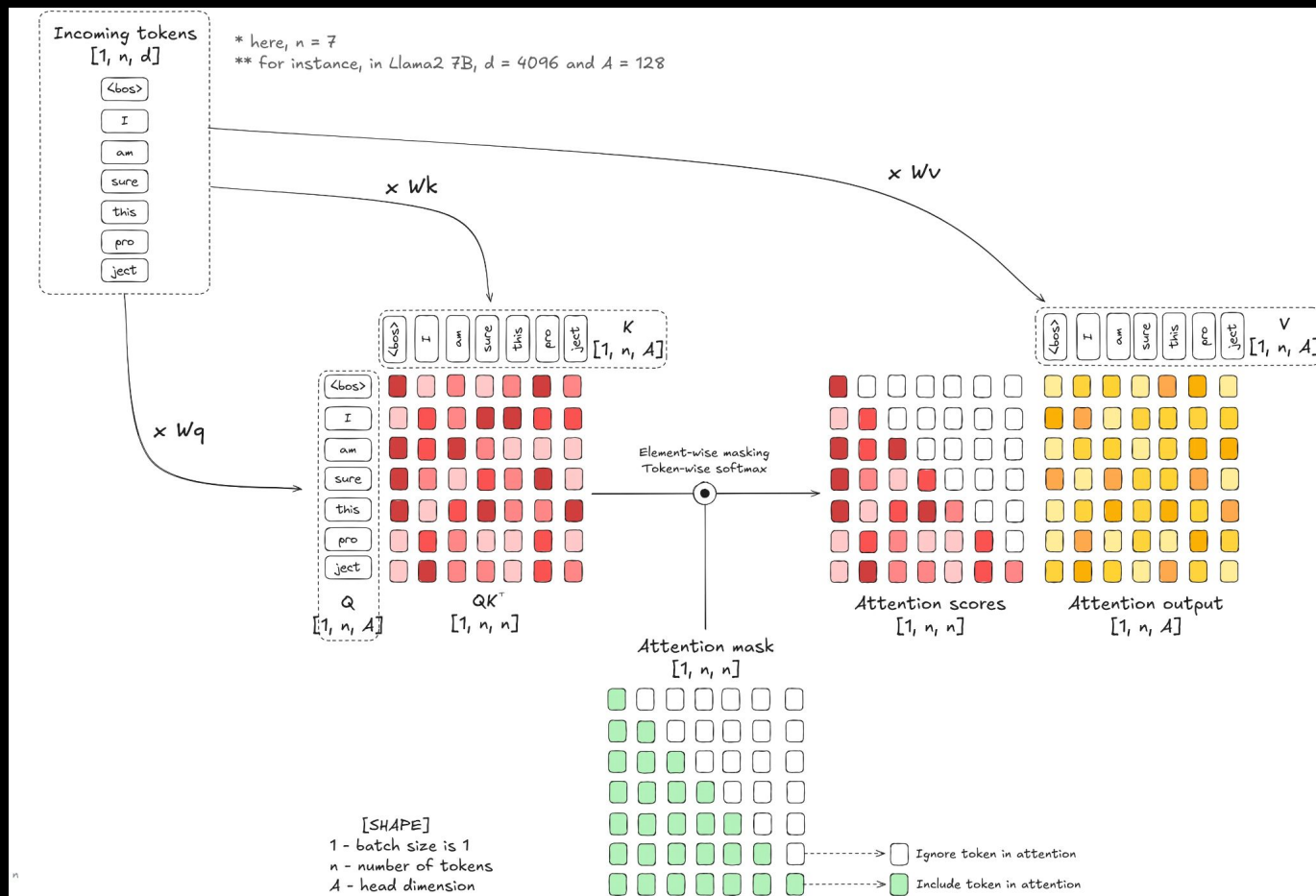


Phases in an LLM inferencing

1. Prefill – KV's are computed of the input tokens and are cached
2. Decoding – Actual next token generation



Computing Attention



Juice

How LLM Requests are served ?

- Request are added to Request Queue
- Scheduler find relevant batch
- Sends to Execution Engine

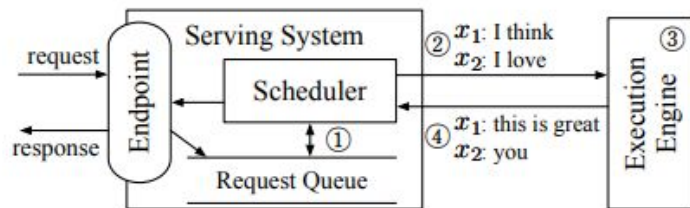


Figure 2: Overall workflow of serving a generative language model with existing systems.

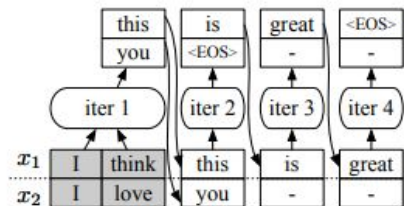


Figure 3: An illustration for a case where the requests have the same input length but some requests finish earlier than others. Shaded tokens represent input tokens. "-" denotes inputs and outputs of extra computation imposed by the scheduling.

Then what is the issue ?

1. Conditions for batching:
 - All the requests in a given batch need to have the same no. of tokens
 - All requests need to be in the same phase ; Prefill/Decoding
(Because of QKV computation.)
2. Requests that have completed processing cannot leave until, all the requests are not completed.
3. New Requests cannot be added instantly, need to wait until previous batches have completed their execution.

Proposed Solution

1. Iterative Scheduling. Requests are batched, and an iteration is executed. Completed requests get to leave, new requests get to join.
2. Selective Batching. Only batch requests that allow for jagged (different shapes) tensors. Ops like Linear, LayerNorm, Activation Layers allow for non-uniform batches.

Selective Batching

What?

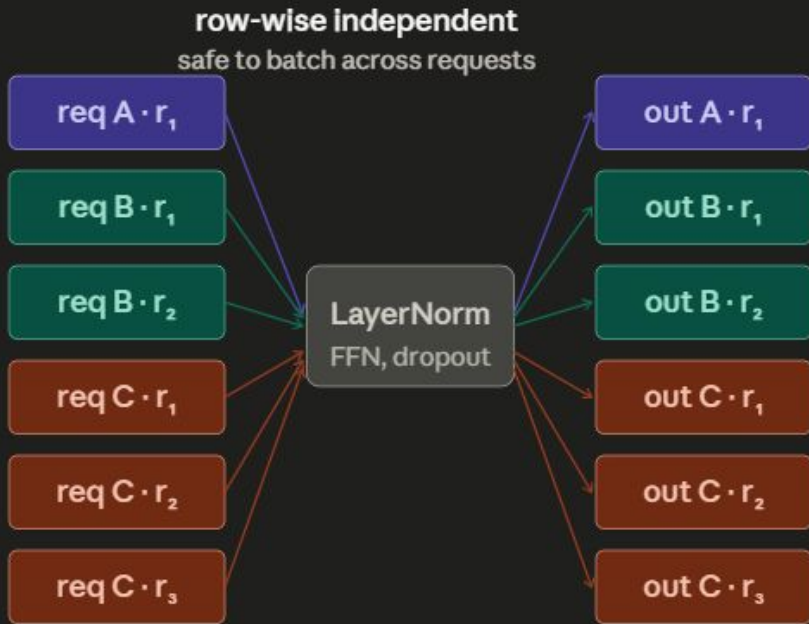
- Batch only operations that are row-wise independent — where computing output row i requires only input row i , with no cross-row dependencies.

How ?

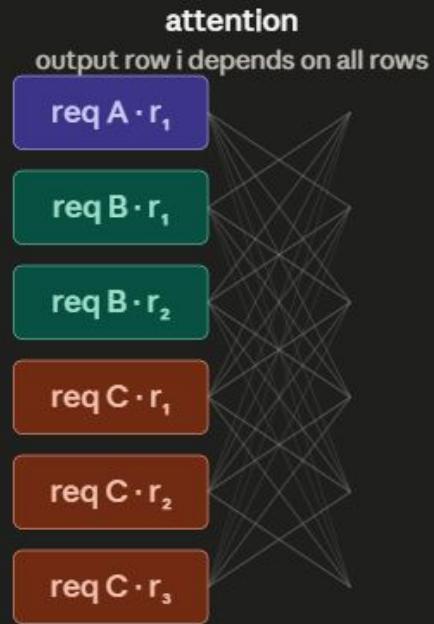
- Flatten Batch Req: 3D tensor [batch, seq_len, h_dim]
to joined 2D Tensor [batch * seq_len, h_dim]
- And Split it back to 3D tensor for non compatible ops like attn, and process them individually.

Compatible Ops: Linear, LayerNorm, Add, and GeLU ...

Selective Batching

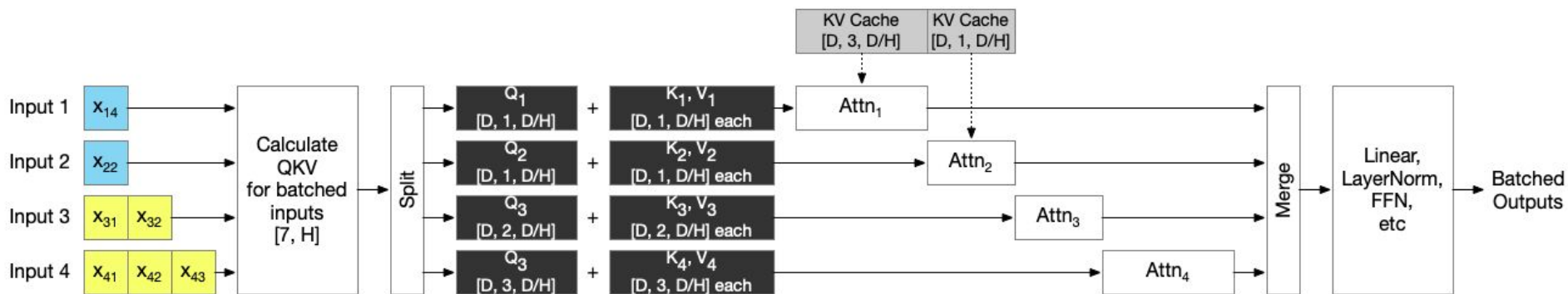


can merge all rows into one call



rows leak across requests
must mask per request

Selective Batching – Implementation



Implement Merge and Split.

Compute non compatible ops unbatched

Iterative Scheduling

Because Selective Scheduling allows jagged input tensors.

- Batch ununiformed/jagged tensors, allowing us to not wait for the matching `n_tokens` requests
- Remove completed requests from the queue and the batch
- Fill up the space in the batches after each iteration - allowing us to start processing new requests.

Because now scheduling is request independent, we can create more aware scheduling algorithms that would require such granularity

Iterative Scheduling – Algorithm

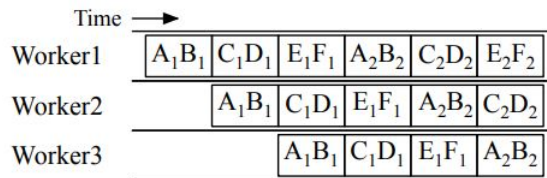
Algorithm 1: ORCA scheduling algorithm

Params: $n_workers$: number of workers, max_bs : max batch size, n_slots : number of K/V slots

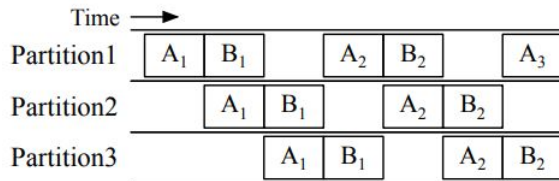
```

1  $n\_scheduled \leftarrow 0$ 
2  $n\_rsrv \leftarrow 0$ 
3 while true do
4    $batch, n\_rsrv \leftarrow \text{Select}(request\_pool, n\_rsrv)$ 
5   schedule engine to run one iteration of
   the model for the batch
6   foreach  $req$  in  $batch$  do
7      $req.state \leftarrow RUNNING$ 
8      $n\_scheduled \leftarrow n\_scheduled + 1$ 
9   if  $n\_scheduled = n\_workers$  then
10    wait for return of a scheduled batch
11    foreach  $req$  in the returned batch do
12       $req.state \leftarrow INCREMENT$ 
13      if  $finished(req)$  then
14         $n\_rsrv \leftarrow n\_rsrv - req.max\_tokens$ 
15         $n\_scheduled \leftarrow n\_scheduled - 1$ 
16
17 def  $Select(pool, n\_rsrv)$ :
18    $batch \leftarrow \{\}$ 
19    $pool \leftarrow \{req \in pool | req.state \neq RUNNING\}$ 
20    $SortByArrivalTime(pool)$ 
21   foreach  $req$  in  $pool$  do
22     if  $batch.size() = max\_bs$  then break
23     if  $req.state = INITIATION$  then
24        $new\_n\_rsrv \leftarrow n\_rsrv + req.max\_tokens$ 
25       if  $new\_n\_rsrv > n\_slots$  then break
26        $n\_rsrv \leftarrow new\_n\_rsrv$ 
27      $batch \leftarrow batch \cup \{req\}$ 
28   return  $batch, n\_rsrv$ 

```



(a) ORCA execution pipeline.



(b) FasterTransformer execution pipeline.

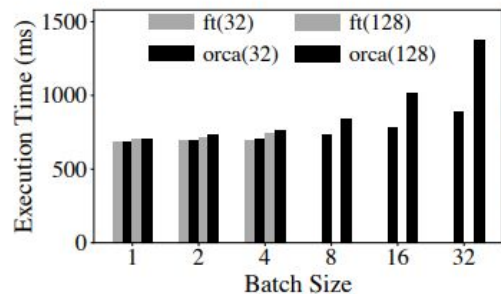
Figure 8: Comparison of the use of pipeline parallelism in ORCA and FasterTransformer where X_i is the i -th iteration of request X .

Note: FastTransformer uses Mini-Batches
– $Req(A, B) \gg (A_1, B_1, A_i, B_i \dots)$

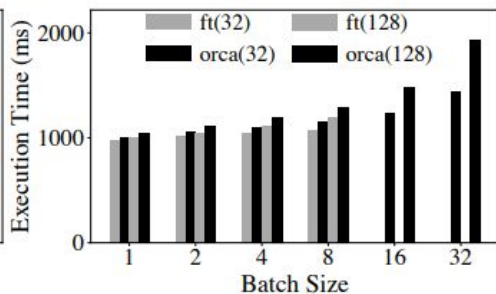
Performance

Paper Evaluates the new Scheduler via:

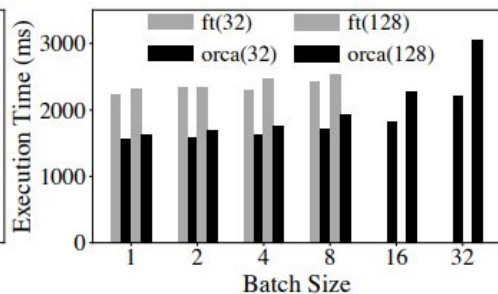
- Comparing with the FastTransformer, another distributed inference engine
- Compares in 2 scenarios:
 - All inputs have same i/o tokens length
 - Real world scenario - varying i/o tokens length



(a) 13B model, 1 GPU.

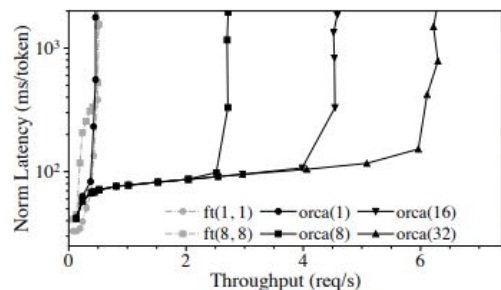


(b) 101B model, 8 GPUs.

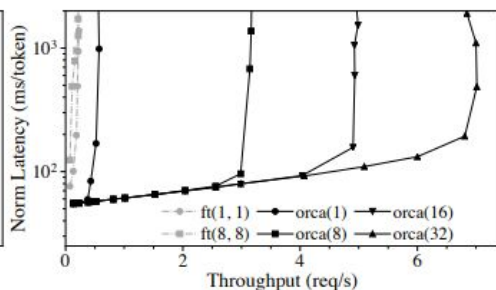


(c) 175B model, 16 GPUs.

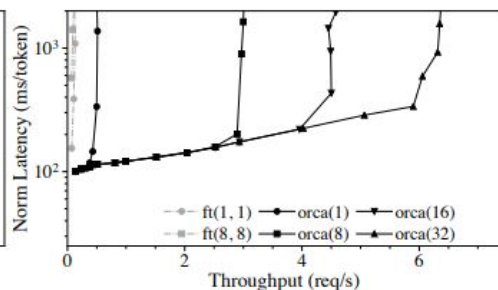
Figure 9: Execution time of a batch of requests using FasterTransformer and the ORCA engine without the scheduling component. Label “ft(n)” represents results from FasterTransformer processing requests with n input tokens. Configurations that incurs out of memory error are represented as missing entries (e.g., ft(32) for the 101B model with a batch size of 16).



(a) 101B model, 8 GPU.



(b) 175B model, 16 GPUs.



(c) 341B model, 32 GPUs.

Figure 10: Median end-to-end latency normalized by the number of generated tokens and throughput. Label “orca(max_bs)” represents results from ORCA with a max batch size of max_bs . Label “ft(max_bs, mbs)” represents results from FasterTransformer with a max batch size of max_bs and a microbatch size of mbs .